

Network Attack Detection Using Machine Learning

Project Development and Evaluation Guidelines

1. Project Objective

The objective of this project is to design, implement, evaluate, and validate a machine learning-based detection system for a selected network threat category. The selected topics include attacks such as DGA Detection, Botnet Detection, Phishing Detection, ARP Spoofing, DNS Spoofing, SSH Brute Force Detection, DDoS Detection, TCP SYN Flood Detection, Cryptojacking Detection, SQL Injection Detection, Ransomware Detection, and other approved topics.

The project must focus on building a solution that can operate beyond laboratory datasets and demonstrate effectiveness when exposed to realistic network traffic. Consequently, the project shall not be evaluated solely on model accuracy but also on experimental rigor, generalization capability, robustness, and deployment feasibility.

2. Dataset Collection and Analysis

The first phase of the project involves identifying and evaluating suitable datasets corresponding to the selected attack category.

Students are required to collect multiple datasets rather than relying on a single source. The purpose of using multiple datasets is to reduce dataset-specific bias and improve the generalizability of the developed solution.

For each dataset, the following analysis must be performed:

- Source and origin of the dataset.
- Attack categories present in the dataset.
- Number of samples.
- Feature description.
- Label quality and reliability.
- Availability of temporal information.
- Presence of duplicate samples.

- Relevance of features to the selected attack.

A dataset shall only be used if its contents are directly relevant to the selected attack category.

For example:

- DGA detection datasets should contain domain names and associated metadata.
- Botnet datasets should contain network flow features.
- ARP Spoofing datasets should contain network packet or flow information associated with ARP behavior.
- SSH brute force datasets should contain authentication-related activity and login attempts.

A justification document explaining the suitability of each selected dataset must be included.

3. Class Distribution Analysis and Data Balancing

Cybersecurity datasets frequently suffer from severe class imbalance. Therefore, prior to model development, a complete class distribution analysis shall be conducted.

The following metrics must be reported:

- Number of benign samples.
- Number of attack samples.
- Class ratio.
- Imbalance severity.

If class imbalance is significant, appropriate balancing techniques may be applied.

Acceptable balancing techniques include:

- Random Oversampling
- Random Undersampling
- SMOTE
- Borderline-SMOTE
- ADASYN
- Hybrid Sampling Approaches
- Cost-sensitive Learning

Students must justify why a particular balancing technique was selected.

Blindly applying balancing techniques is prohibited. The effect of balancing on model performance shall be analyzed and documented.

4. Temporal-Aware Data Splitting

One of the most common mistakes in cybersecurity machine learning research is information leakage caused by improper dataset splitting.

To ensure realistic evaluation, temporal ordering must be preserved throughout experimentation.

Under no circumstances should random shuffling be performed before train-test splitting.

Similarly, Stratified Sampling and Stratified Cross Validation must not be used when they destroy temporal relationships between observations.

The dataset shall be divided according to temporal order as follows:

- Training Set: 70%
- Validation Set: 10%
- Testing Set: 20%

The oldest observations shall be allocated to the training set while newer observations shall be reserved for validation and testing.

The final testing dataset must always represent future unseen observations relative to the training period.

Students must provide evidence that temporal ordering was maintained.

5. Prevention of Information Leakage

A primary requirement of this project is the elimination of information leakage.

The following activities are prohibited:

- Computing normalization statistics using the complete dataset.
- Performing feature selection before train-test separation.
- Applying balancing techniques before train-test separation.

- Using future information during feature engineering.
- Including duplicate samples across train and test datasets.

All preprocessing operations must be learned exclusively from the training data and subsequently applied to validation and testing datasets.

A dedicated section documenting leakage prevention measures must be included.

6. Model Selection and Development

Each project must evaluate a minimum of three machine learning models.

Model selection shall be driven by the characteristics of the problem and dataset rather than convenience.

Random selection of algorithms is not acceptable.

Students must provide technical justification explaining why each model is suitable for the chosen attack type.

Examples include:

- Random Forest
- XGBoost
- LightGBM
- CatBoost
- LSTM
- GRU
- Temporal Convolutional Networks
- Isolation Forest
- One-Class SVM
- Autoencoders
- Graph-Based Models
- Transformer-Based Models

The first model should act as a baseline.

The second and third models should be selected based on the limitations identified in the baseline model and their expected advantages for the target problem.

For each model, students must explain:

- Architecture

- Training procedure
 - Hyperparameters
 - Computational requirements
 - Expected strengths
 - Expected weaknesses
-

7. Comparative Evaluation

A complete comparison shall be conducted among all developed models.

The following metrics must be reported:

- Accuracy
- Precision
- Recall
- F1 Score
- ROC-AUC
- PR-AUC
- False Positive Rate
- False Negative Rate
- Detection Latency
- Training Time
- Inference Time

Confusion matrices must be provided for every model.

The objective is not merely to identify the best-performing model but also to understand why performance differences occur.

8. Generalization Study Using Unseen Datasets

A major limitation of cybersecurity machine learning systems is overfitting to a specific dataset.

To evaluate generalization capability, an additional independent dataset must be collected.

This dataset must not contribute any samples to the training, validation, or testing stages.

The workflow shall therefore include:

Dataset A:

- Training (70%)
- Validation (10%)
- Testing (20%)

Dataset B:

- External Generalization Dataset

The final trained model shall be evaluated on Dataset B without retraining.

Performance degradation shall be analyzed and discussed.

This section is mandatory and will significantly influence project evaluation.

9. Ablation Study

An ablation study is one of the most important components of this project.

The objective is to understand the behavior of the system and identify the contribution of individual components.

Students shall systematically remove or modify components and observe the impact on performance.

Possible ablation experiments include:

- Removing selected features.
- Removing balancing strategies.
- Changing temporal windows.
- Altering feature engineering pipelines.
- Modifying hyperparameters.
- Comparing packet-level versus flow-level features.
- Evaluating different feature groups independently.

For each experiment, students must answer:

- What was changed?
- Why was it changed?
- What was the observed impact?

- What does this reveal about the model?

The discussion should focus on understanding model behavior rather than merely reporting metric changes.

10. Real-Time Detection Adapter Development

Dataset evaluation alone is insufficient to demonstrate practical applicability.

Students must therefore develop a real-time inference adapter.

The adapter shall:

1. Capture real network traffic.
2. Extract the features required by the trained model.
3. Transform raw traffic into model-compatible input.
4. Perform real-time inference.
5. Generate attack detection alerts.

The adapter acts as the bridge between real-world traffic and the trained machine learning model.

A detailed architecture diagram describing this pipeline must be included.

11. Real Traffic Generation and Attack Emulation

To evaluate real-world performance, realistic traffic must be generated.

Traffic generation may be performed using:

- Npcap
- Wireshark
- Scapy
- Tcpdump
- Custom Traffic Generators

- Virtual Test Environments

Students shall develop scripts that emulate the selected attack category.

Examples include:

- DNS spoofing scripts.
- ARP spoofing scripts.
- SSH brute force automation.
- SYN flood generators.
- DDoS simulation environments.
- DGA traffic generation scripts.

The attack generation process must be documented and reproducible.

12. Real-Time Evaluation

The trained system shall be tested under live traffic conditions.

Evaluation metrics shall include:

- Real-time Accuracy
- Precision
- Recall
- F1 Score
- False Positive Rate
- False Negative Rate
- Detection Delay
- CPU Utilization
- Memory Consumption
- Throughput
- Inference Latency

Students must compare:

1. Dataset Performance
2. External Dataset Performance
3. Real Traffic Performance

Any discrepancies must be analyzed.

13. Expected Deliverables

At project completion, each team shall submit:

1. Literature Survey
 2. Dataset Analysis Report
 3. Data Balancing Report
 4. Temporal Splitting Documentation
 5. Leakage Prevention Documentation
 6. Baseline Model Implementation
 7. Two Advanced Model Implementations
 8. Comparative Evaluation Report
 9. Generalization Study Report
 10. Complete Ablation Study
 11. Real-Time Traffic Adapter
 12. Attack Emulation Scripts
 13. Experimental Results
 14. Final Technical Report
 15. Source Code Repository
 16. Deployment and Execution Guide
-

14. Final Evaluation Philosophy

The primary objective of this project is not to achieve the highest accuracy on a benchmark dataset.

A model achieving 99% accuracy on a dataset but failing on unseen traffic demonstrates limited practical value.

A successful project is one that:

- Preserves temporal integrity.
- Avoids information leakage.
- Demonstrates strong generalization.
- Provides explainable observations through ablation studies.
- Performs reliably on real-world traffic.
- Can be deployed as a practical cybersecurity detection system.

The final report should therefore emphasize scientific rigor, experimental transparency, reproducibility, and real-world applicability rather than focusing exclusively on benchmark performance metrics.

